교육부 Ministry of Education

NRF 한국연구재단 National Research Foundation of Korea

공유교과목 딥러닝

케라스 활용

참조: 케라스 공식 문서 (https://keras.io/)

Convergence and Open Sharing System COSS 인공지능
전남대학교 CHONNAM NATIONAL UNIVERSITY
성균관대학교 SUNGKYUNKWAN UNIVERSITY
KNU 경북대학교 KYUNGPOOK NATIONAL UNIVERSITY
전주대학교 JEONJU UNIVERSITY
서울시립대학교 UNIVERSITY OF SEOUL
서울과학기술대학교 SEOUL NATIONAL UNIVERSITY OF SCIENCE & TECHNOLOGY
영진전문대학교 YEUNGJIN UNIVERSITY

Contents

- 시작하기: 케라스 기반의 딥러닝 예제
- 케라스 제공 라이브러리 overview

시작하기: 케라스 기반의 딥러닝 예제

케라스 활용

- **케라스 기반의 딥러닝 예제**
  - 딥러닝 구조 생성 및 학습
    - ✓ 모델 초기화
      : sequential 모델은 각 레이어에 하나의 입력 텐서와 하나의 출력텐서가 있는 일반 레이어 스택에 적합한 모델

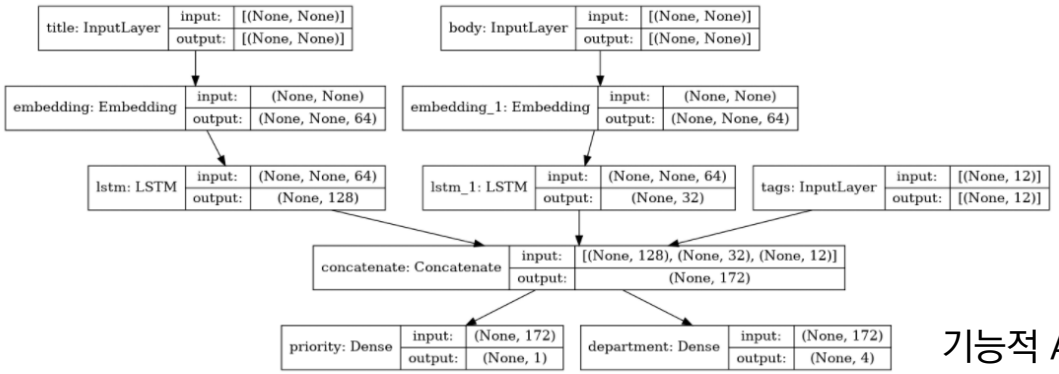
```
from keras.models import Sequential

model = Sequential()
```

```
Model: "sequential_6"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 123, 123, 32)      2432
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 121, 121, 32)      9248
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 40, 40, 32)        0
=================================================================
Total params: 11,680
Trainable params: 11,680
Non-trainable params: 0
```

순차 모델 예시

    - ✓ 만약 sequential 보다 더욱 유연한 모델을 생성하고자 하면 functional API 를 활용 (https://keras.io/guides/functional_api/)



기능적 API 예시

케라스 활용

- **케라스 기반의 딥러닝 예제**
  - 딥러닝 구조 생성 및 학습
    - ✓ 레이어 쌓기
      - model.add를 통해 다음과 같이 원하는 형태의 layer (층)을 쌓을 수 있음
      - Input tensor의 크기 (input_dim), 활성화 함수 ('relu', 'softmax', 'sigmoid'),
      - Output tensor (units) 를 설정

```
from keras.layers import Dense

model.add(Dense(units=64, activation='relu', input_dim=100))
model.add(Dense(units=10, activation='softmax'))
```

    - ✓ 학습 과정 설정
      - Loss function, optimizer, metric등 학습에 필요한 기법설정 가능

```
model.compile(loss='categorical_crossentropy',
              optimizer='sgd',
              metrics=['accuracy'])
```

```
model.compile(loss=keras.losses.categorical_crossentropy,
              optimizer=keras.optimizers.SGD(lr=0.01, momentum=0.9, nesterov=True))
```

▲ 위와 같이 각 기법들의 세부 파라미터 조정 가능

케라스 활용

- **케라스 기반의 딥러닝 예제**
  - 딥러닝 구조 생성 및 학습
    - ✓ 레이어 학습
      - model.fit을 통해 간편하게 학습 가능
      - 학습 데이터, 학습 레이블, epoch, batch_size 등을 입력
      - 기호에 맞게 epoch, batch_size 값을 설정

```
# x_train and y_train are Numpy arrays --just like in the Scikit-Learn API.
model.fit(x_train, y_train, epochs=5, batch_size=32)
```

    - ✓ 학습 성능 평가
      - 기존 test 데이터에 대한 성능 평가

```
loss_and_metrics = model.evaluate(x_test, y_test, batch_size=128)
```

      - 새로운 unseen 데이터에 대한 학습 모델의 예측 값 출력

```
classes = model.predict(x_test, batch_size=128)
```

케라스 제공 라이브러리 overview

케라스 활용

- 제공 라이브러리 overview
  - 손실함수
    - ✓ https://keras.io/api/losses/

Probabilistic losses

- BinaryCrossentropy class
- CategoricalCrossentropy class
- SparseCategoricalCrossentropy class
- Poisson class
- binary_crossentropy function
- categorical_crossentropy function
- sparse_categorical_crossentropy function
- poisson function
- KLDivergence class
- kl_divergence function

[케라스에서 제공되는 손실함수 리스트]

Regression losses

- MeanSquaredError class
- MeanAbsoluteError class
- MeanAbsolutePercentageError class
- MeanSquaredLogarithmicError class
- CosineSimilarity class
- mean_squared_error function
- mean_absolute_error function
- mean_absolute_percentage_error function
- mean_squared_logarithmic_error function
- cosine_similarity function
- Huber class
- huber function
- LogCosh class
- log_cosh function

Hinge losses for "maximum-margin" classification

- Hinge class
- SquaredHinge class
- CategoricalHinge class
- hinge function
- squared_hinge function
- categorical_hinge function

케라스 활용

- **제공 라이브러리 overview**
  - Optimizer
    - ✓ https://keras.io/api/optimizers/

[케라스에서 제공되는 최적화 알고리즘]

- SGD
- RMSprop
- Adam
- Adadelta
- Adagrad
- Adamax
- Nadam
- Ftrl

```
>>> opt = tf.keras.optimizers.Adam(learning_rate=0.1)
>>> var1 = tf.Variable(10.0)
>>> loss = lambda: (var1 ** 2)/2.0       # d(loss)/d(var1) == var1
>>> step_count = opt.minimize(loss, [var1]).numpy()
>>> # The first step is `-learning_rate*sign(grad)`
>>> var1.numpy()
9.9
```

[Adam optimizer 활용 예]

케라스 활용

- 제공 라이브러리 overview
  - Metrics
    - ✓ https://keras.io/api/metrics/

Accuracy metrics

- Accuracy class
- BinaryAccuracy class
- CategoricalAccuracy class
- SparseCategoricalAccuracy class
- TopKCategoricalAccuracy class
- SparseTopKCategoricalAccuracy class

Probabilistic metrics

- BinaryCrossentropy class
- CategoricalCrossentropy class
- SparseCategoricalCrossentropy class
- KLDivergence class
- Poisson class

Regression metrics

- MeanSquaredError class
- RootMeanSquaredError class
- MeanAbsoluteError class
- MeanAbsolutePercentageError class
- MeanSquaredLogarithmicError class
- CosineSimilarity class
- LogCoshError class

Classification metrics based on True/False positives & negatives

- AUC class
- Precision class
- Recall class
- TruePositives class
- TrueNegatives class
- FalsePositives class
- FalseNegatives class
- PrecisionAtRecall class
- SensitivityAtSpecificity class
- SpecificityAtSensitivity class

Image segmentation metrics

- MeanIoU class

Hinge metrics for "maximum-margin" classification

- Hinge class
- SquaredHinge class
- CategoricalHinge class

[케라스에서 제공되는 metrics]

케라스 활용

- 제공 라이브러리 overview
  - Metrics
    - ✓ https://keras.io/api/metrics/
    - ✓ complie() 함수의 인자로 설정하여 학습 과정에서 metric 활용 가능
    - ✓ 여러 개의 metric 병렬 활용 가능
    - ✓ fit() 함수를 통해 학습되는 도중 해당 metric 이 활용 됨

```
model.compile(
    optimizer='adam',
    loss='mean_squared_error',
    metrics=[
        metrics.MeanSquaredError(name='my_mse'),
        metrics.AUC(name='my_auc'),
    ]
)
```

```
model.compile(
    optimizer='adam',
    loss='mean_squared_error',
    metrics=[
        'MeanSquaredError',
        'AUC',
    ]
)
```

    - ✓ 아래와 같이 metric 함수 단독 활용도 가능

```
m = tf.keras.metrics.AUC()
m.update_state([0, 1, 1, 1], [0, 1, 0, 0])
print('Intermediate result:', float(m.result()))

m.update_state([1, 1, 1, 1], [0, 1, 1, 0])
print('Final result:', float(m.result()))
```

케라스 활용

- 제공 라이브러리 overview
  - 데이터 로드
    - ✓ https://keras.io/api/data_loading/

```
from tensorflow import keras

train_ds = keras.utils.image_dataset_from_directory(
    directory='training_data/',
    labels='inferred',
    label_mode='categorical',
    batch_size=32,
    image_size=(256, 256))
validation_ds = keras.utils.image_dataset_from_directory(
    directory='validation_data/',
    labels='inferred',
    label_mode='categorical',
    batch_size=32,
    image_size=(256, 256))

model = keras.applications.Xception(
    weights=None, input_shape=(256, 256, 3), classes=10)
model.compile(optimizer='rmsprop', loss='categorical_crossentropy')
model.fit(train_ds, epochs=10, validation_data=validation_ds)
```

[데이터 로드 사용 예시]

Image data loading

- image_dataset_from_directory function
- load_img function
- img_to_array function

Timeseries data loading

- timeseries_dataset_from_array function

Text data loading

- text_dataset_from_directory function

[케라스에서 활용 가능한 데이터 로더]

케라스 활용

- **제공 라이브러리 overview**
  - 데이터 로드 (기본 제공 공개 데이터셋)
    - ✓ https://keras.io/api/datasets/

MNIST digits classification dataset

- load_data function

CIFAR10 small images classification dataset

- load_data function

CIFAR100 small images classification dataset

- load_data function

IMDB movie review sentiment classification dataset

- load_data function
- get_word_index function

Reuters newswire classification dataset

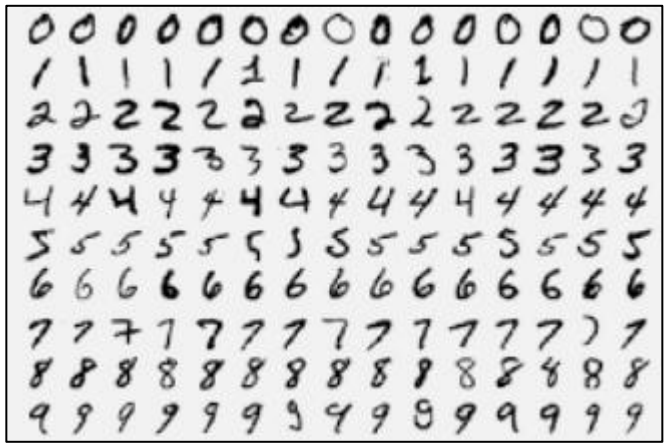
- load_data function
- get_word_index function

Fashion MNIST dataset, an alternative to MNIST

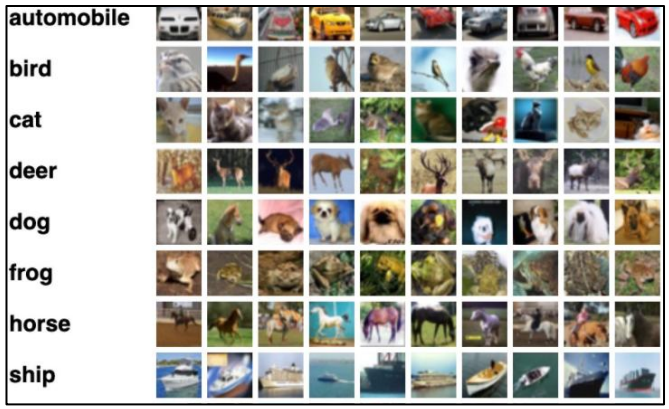
- load_data function

Boston Housing price regression dataset

- load_data function



[필기 인식 데이터 MNIST]



[영상 분류 데이터 CIFAR100]

케라스 활용

- 제공 라이브러리 overview
  - 네트워크 구조
    - ✓ https://keras.io/api/applications/
    - ✓ 다양한 검증된 네트워크 구조 및 모델들 활용 가능

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	0.790	0.945	22,910,480	126	109.42	8.06
VGG16	528	0.713	0.901	138,357,544	23	69.50	4.16
VGG19	549	0.713	0.900	143,667,240	26	84.75	4.38
ResNet50	98	0.749	0.921	25,636,712	-	58.20	4.55
ResNet101	171	0.764	0.928	44,707,176	-	89.59	5.19
ResNet152	232	0.766	0.931	60,419,944	-	127.43	6.54
ResNet50V2	98	0.760	0.930	25,613,800	-	45.63	4.42
ResNet101V2	171	0.772	0.938	44,675,560	-	72.73	5.43
ResNet152V2	232	0.780	0.942	60,380,648	-	107.50	6.64
InceptionV3	92	0.779	0.937	23,851,784	159	42.25	6.86
InceptionResNetV2	215	0.803	0.953	55,873,736	572	130.19	10.02
MobileNet	16	0.704	0.895	4,253,864	88	22.60	3.44
MobileNetV2	14	0.713	0.901	3,538,984	88	25.90	3.83

Model
DenseNet121
DenseNet169
DenseNet201
NASNetMobile
NASNetLarge
EfficientNetB0
EfficientNetB1
EfficientNetB2
EfficientNetB3
EfficientNetB4
EfficientNetB5
EfficientNetB6
EfficientNetB7

케라스 활용

- 제공 라이브러리 overview
  - 네트워크 구조
    - ✓ https://keras.io/api/applications/
    - ✓ 사용 예시 : Xception function

```
tf.keras.applications.Xception(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
    classifier_activation="softmax",
)
```

VGG19

```
tf.keras.applications.VGG19(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
    classifier_activation="softmax",
)
```

    - ✓ 각 네트워크 구성 요소에 맞도록 초기화 및 활용

교육부 Ministry of Education

NRF 한국연구재단 National Research Foundation of Korea

thank you

본 과제(결과물)는 교육부와 한국연구재단의 재원으로 지원을 받아 수행된
디지털신기술인재양성 혁신공유대학사업의 연구결과입니다.

Convergence and Open Sharing System COSS 인공지능 · 전남대학교 CHONNAM NATIONAL UNIVERSITY · 성균관대학교 SUNGKYUNKWAN UNIVERSITY · KNU 경북대학교 KYUNGPOOK NATIONAL UNIVERSITY · 전주대학교 JEONJU UNIVERSITY · 서울시립대학교 UNIVERSITY OF SEOUL · 서울과학기술대학교 SEOUL NATIONAL UNIVERSITY OF SCIENCE & TECHNOLOGY · 영진전문대학교 YEUNGJIN UNIVERSITY